



Inovação no
Ensino Superior
para a Indústria

SEMANA 1

Para onde o mercado mobile aponta

Panorama do mercado e o ecossistema Android: quem usa, quem controla a plataforma e o que isso muda no código que a gente vai escrever.

Desenvolvimento de Aplicações para Dispositivos Móveis · Prof. Nisflei · 2026

Roteiro

- 1 Onde estão os usuários, no Brasil e no mundo
- 2 Volume de um lado, dinheiro do outro
- 3 Quem controla o ecossistema Android
- 4 As camadas da plataforma
- 5 Fragmentação e o número que decide o projeto
- 6 Nativo, híbrido e multiplataforma
- 7 Java, Kotlin e a virada do Compose
- 8 O que a gente constrói neste semestre

78,96%

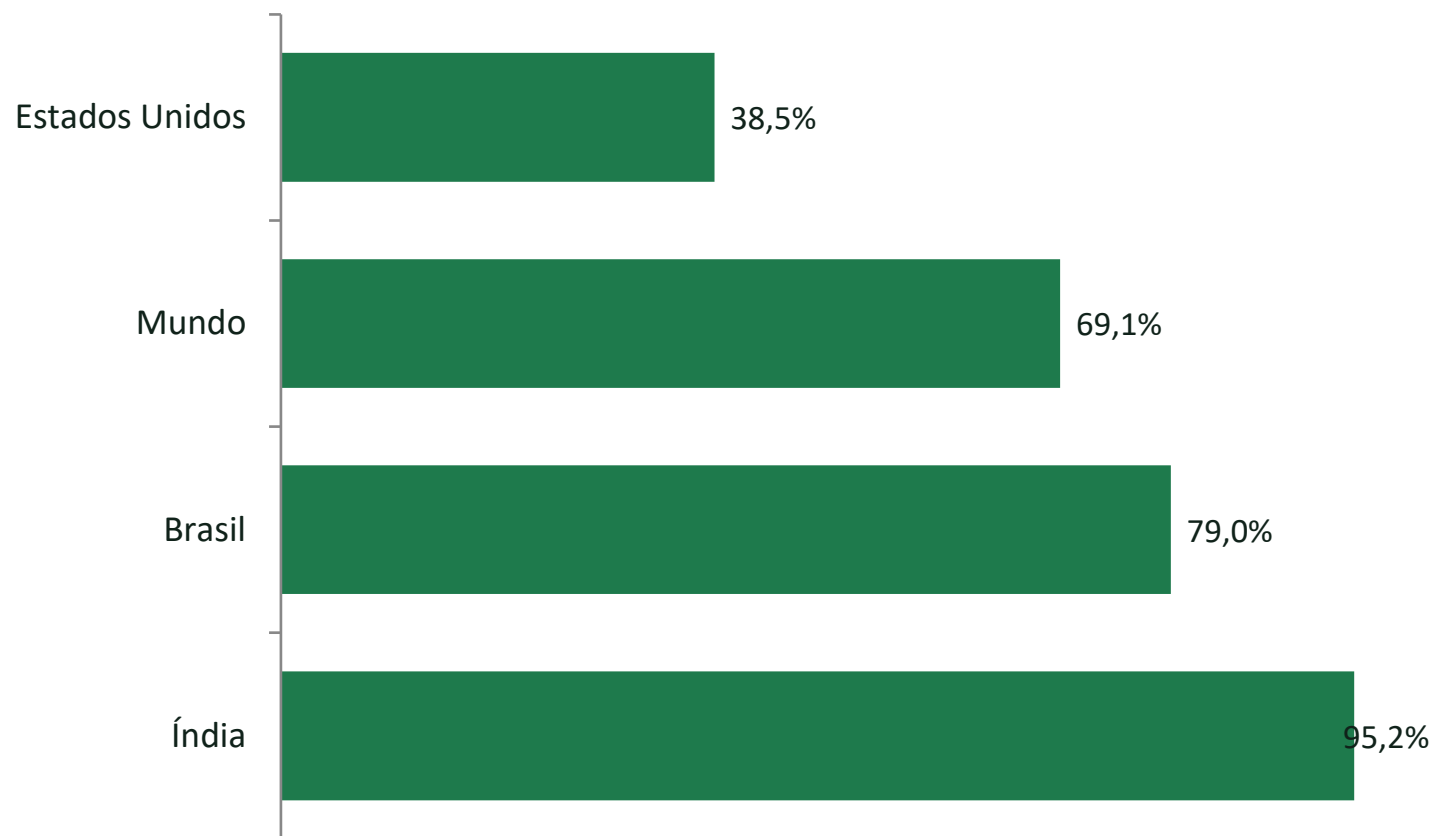
dos acessos por celular no Brasil vêm de aparelhos Android

O iOS fica com 21%. Se um cliente brasileiro só tem orçamento para uma plataforma, a conversa acaba rápido.

O mesmo dado, em outros lugares

Índia	95,2%
Brasil	79,0%
Mundo	69,1%
Estados Unidos	38,5%

Android não é dominante em toda parte



Por que isso importa agora

Um app pensado para o mercado brasileiro roda, na prática, em aparelho Android de entrada, com tela pequena, memória curta e rede instável.

É para esse aparelho que vamos testar o projeto do semestre, não para o emulador rodando com 8 GB de RAM.

Android tem o público. iOS tem o cartão.

Android

- Cerca de 3,9 bilhões de aparelhos ativos no mundo.
- Google Play respondeu por perto de 69% dos 149 bilhões de downloads de 2025.
- Alcance alto, ticket baixo. O dinheiro vem de anúncio, comércio e serviço, não de assinatura premium.

iOS

- Menos de 30% dos aparelhos no mundo.
- Fica com cerca de 64% de tudo que o consumidor gasta dentro de aplicativos.
- Concentrado em Estados Unidos, Japão, Reino Unido e norte da Europa.

As duas afirmações são verdadeiras ao mesmo tempo. Quem decide plataforma olhando só uma delas erra o modelo de negócio do app.

Android é aberto, mas não inteiro

1

AOSP

Android Open Source Project

O sistema em si, sob licença Apache. Qualquer um baixa, compila e instala. É o que a Amazon fez no Fire OS.

2

Fabricantes

Samsung, Motorola, Xiaomi

Pegam o AOSP, colocam a interface própria e decidem por quanto tempo o seu aparelho vai receber atualização.

3

Google Play Services

Peça fechada

Mapas, login, notificação push, pagamento. Não é código aberto e depende de licença com o Google.

O caso Huawei mostrou o tamanho da diferença: os celulares continuaram rodando Android, e mesmo assim ficaram sem Play Store, sem Gmail e sem mapa do Google. Aberto não quer dizer independente.

O que existe entre o seu código e o hardware

Seu aplicativo

Activities, layouts, a lógica que você escreve

← Você programa aqui em cima

Java API Framework

Activity Manager, View System, Content Providers

Bibliotecas nativas e ART

SQLite, OpenGL, e a máquina virtual que roda seu código

Camada de abstração de hardware (HAL)

Interface padrão para câmera, sensores, áudio, Bluetooth

Kernel Linux

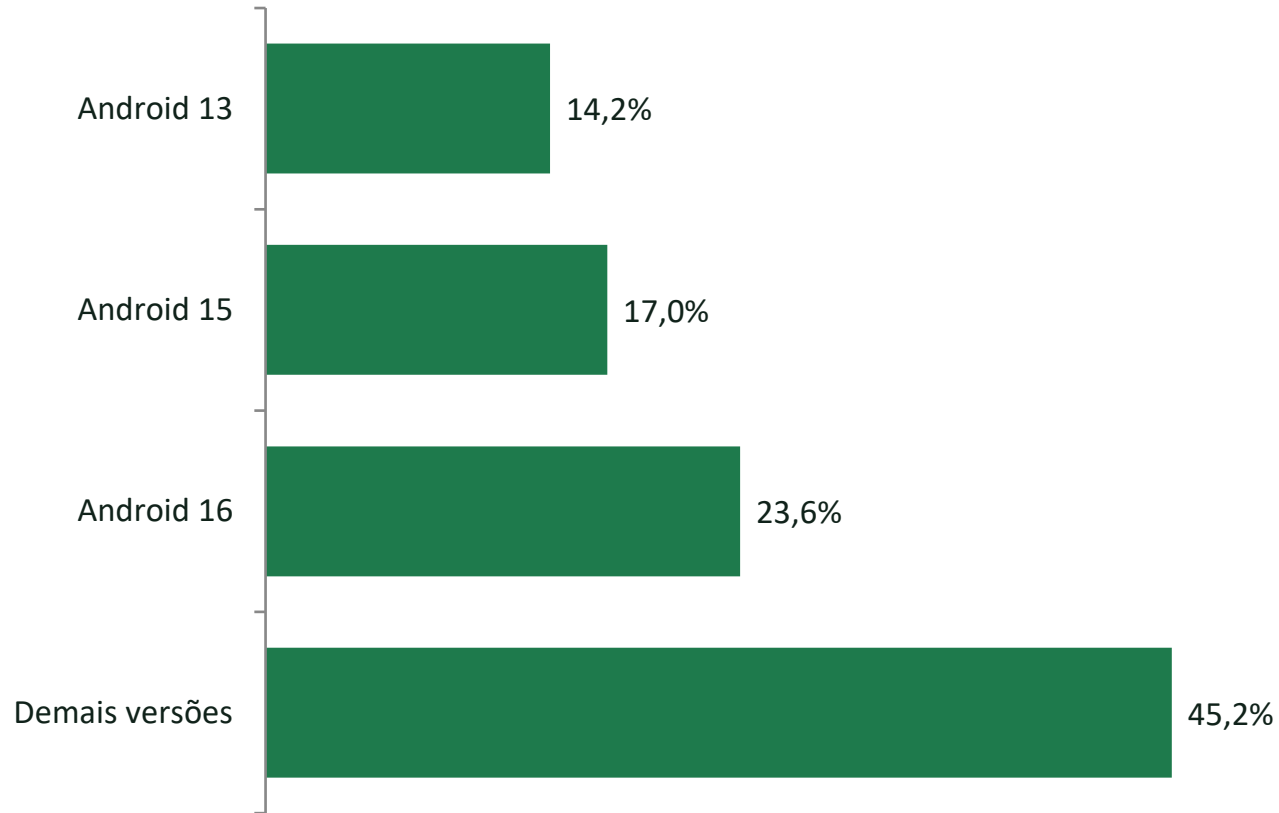
Processos, memória, drivers, energia

Nenhuma linha do seu app fala direto com a câmera.

Você pede ao framework, o framework desce a pilha, e o kernel resolve.

É por isso que o mesmo código funciona num Motorola e num Samsung.

A versão mais popular tem menos de um quarto do mercado



86,9%

é o alcance de um app com minSdk 30. Esse é o número que a gente vai configurar no build.gradle na semana 2.

A comparação incômoda

Um ano depois do lançamento, o iOS 18 já estava em 84% dos iPhones. O Android 16 chegou a 23,6% no mesmo intervalo. Atualização de sistema depende do fabricante, e ele não tem pressa.

Quatro caminhos, um resultado parecido

Nativo Android

Kotlin ou Java

Acesso direto a tudo que a plataforma oferece, no dia em que ela oferece. Custo: uma base de código por sistema.

Flutter

Dart

Desenha a própria interface. Rende protótipo rápido e visual idêntico nos dois sistemas, com um runtime pesado junto.

React Native

JavaScript

Aproveita time de web. Componentes nativos de verdade, mas a ponte entre os dois mundos custa em desempenho e em bug estranho.

Kotlin Multiplatform

Kotlin

Compartilha regra de negócio e deixa cada interface nativa. É a aposta atual do Google e da JetBrains.

A briga sobre qual é o melhor rende pouco. Todos terminam num APK rodando sobre a mesma pilha, e quem entende o nativo consegue depurar qualquer um dos outros três. Por isso a disciplina começa pelo nativo.

Por que este curso pode começar em Java 🤔

O que o Google anunciou

- Kotlin é a linguagem oficial desde 2019.
- Com o Android 17, em junho de 2026, o desenvolvimento passou a ser Compose-first: as APIs novas saem só para Jetpack Compose.

O que isso muda para você

- Ciclo de vida, Intent, permissão e persistência são os mesmos nas duas linguagens. Aprender isso em Java não é tempo perdido.
- Você já sabe Java de Programação Orientada a Objetos, e vamos gastar o semestre com Android, não com sintaxe nova.
- Aplicativo em produção com XML e RecyclerView existe aos milhares, e alguém precisa dar manutenção.
- Kotlin entra nas últimas semanas, já com o modelo mental pronto.

Desmonte um app que você usa todo dia

Em dupla, vinte minutos. Peguem o celular e escolham um app instalado.

- 1 Que recursos do aparelho ele usa? Câmera, GPS, notificação, biometria, contatos.
 - 2 O que aparece na tela quando você desliga a internet? Ele guarda algo localmente?
 - 3 Onde moram os dados: no aparelho, no servidor, ou nos dois ao mesmo tempo?
 - 4 Como esse app ganha dinheiro?
-

Por onde eu começo?

3 Passos para Iniciar Hoje

1

Escolha sua Plataforma

Decida entre iOS (Swift) ou Android (Kotlin). Não tente aprender os dois ao mesmo tempo.

2

Instale a IDE

Baixe o Android Studio ou Xcode. Configure seu ambiente de desenvolvimento.

3

Crie o "Hello World"

Siga um tutorial oficial e faça seu primeiro app rodar no emulador. A prática vence a teoria.

Dica de Ouro

"O melhor código é aquele que você escreve. Comece pequeno, mas comece agora."



Co
Tip
Beg

A vida como ela é... Estratégia MVP

Minimum Viable Product: Lance Rápido, Aprenda Rápido

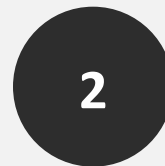
O que é MVP?

A versão mais simples de um aplicativo que resolve o problema principal do usuário, lançada para validar a ideia com o mínimo de esforço e custo.



Construir

Foque apenas nas funcionalidades essenciais (Core).



Medir

Lance para usuários reais e colete dados de uso.



Aprender

Analise o feedback e melhore o produto (Iteração).

Redução de Risco

Evite gastar meses criando algo que ninguém quer. Valide a demanda primeiro.

Velocidade

Reduza o tempo de lançamento de 6-12 meses para 2-3 meses.

Foco no Usuário

Construa o que os usuários realmente precisam, não o que você acha que eles querem.

Publicando sue App

Apple App Store vs Google Play Store

Critério	Apple App Store	Google Play Store
Custo Anual	\$99 / ano	\$25 (pagamento único)
Tempo de Revisão	3-7 dias (manual)	Horas a 2 dias (automatizada)
Taxa de Aprovação	~60% (primeira tentativa)	~85% (primeira tentativa)
Rigor	Muito rigoroso (design, privacidade)	Moderado (foco em segurança)
Atualizações	Cada update passa por revisão	Revisão rápida, rollout gradual
Alcance Global	175 países	150+ países

Google Play

Processo mais rápido e flexível que iOS.

40%

dos apps são rejeitados na primeira submissão (bugs, privacidade, design). Apple Store

O caminho até a loja

2,4 mi

aplicativos disponíveis no Google Play

2,5 bi

usuários ativos por mês na loja

US\$ 25

taxa única para abrir conta de desenvolvedor

15%

comissão do Google sobre o primeiro milhão de dólares por ano

Publicar não é o fim do trabalho

- A loja exige política de privacidade, formulário de segurança de dados e classificação indicativa antes de aceitar o envio.
- Desde 2021 o formato de envio é o Android App Bundle, e não mais o APK. O Google gera o APK final para cada aparelho.
- Revisão leva de horas a alguns dias, e reprovação por permissão mal justificada é comum. Vamos passar por isso na semana 19.

O que você vai ter construído em vinte semanas

1

Semanas 1 a 6

Fundamentos

Ambiente, Activity, ciclo de vida, layouts, navegação e Fragments

2

Semanas 7 a 12

Dados

RecyclerView, SQLite com Room, threads e consumo de API REST

3

Semanas 13 a 17

Nuvem e recursos

Firebase, Cloudinary com câmera, GPS e um assistente com Gemini

4

Semanas 18 a 20

Entrega

MVVM, testes com JUnit, geração do AAB e publicação

O projeto

Um aplicativo que consome uma API pública, guarda dados no aparelho, sobe foto para a nuvem e sai publicável.

A escolha do tema é sua. Traga uma ideia na semana 3.

Fontes dos dados



StatCounter GlobalStats

gs.statcounter.com — participação de sistemas operacionais e versões do Android, por país e por mês



Blog Android Developers

android-developers.googleblog.com — anúncio do Android 17, junho de 2026



Sensor Tower, State of Mobile 2026

downloads e gasto do consumidor em Google Play e App Store



IDC Worldwide Quarterly Mobile Phone Tracker

unidades enviadas por trimestre e por fabricante



Documentação oficial da plataforma

developer.android.com — arquitetura, níveis de API e regras do Play Console

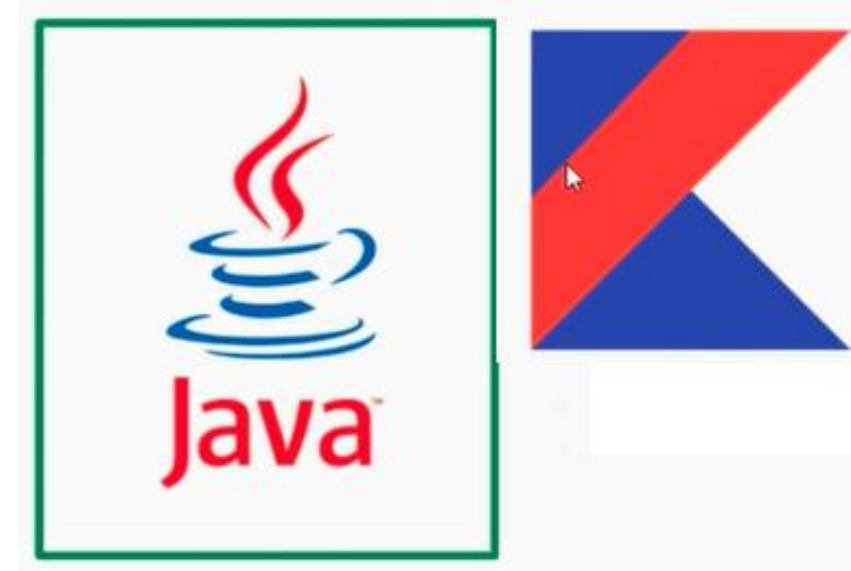
Número de mercado envelhece rápido. Antes de repetir qualquer porcentagem deste material em trabalho ou entrevista, abra a fonte e confira o mês.

Hands On



O que precisamos para começar...

- Android Studio
 - Android SDK
 - Linguagem
 - Java ou Kotlin
-
- *Download:*
 - <https://developer.android.com/studio>





Inovação no
Ensino Superior
para a Indústria